# ENDG 510 L01 - (FALL 2026)

# Lab Manual (Experiment 2)

Instructor: Dr. Hadis Karimipour

# Lab Tasks

**TASK:** Prototyping and Live Data Visualization in Cyber-Physical Systems (CPS)

**Objective:** To understand how to design CPS using virtual prototyping and its subsequent visualization for real-time monitoring, as shown in Fig 1.

**Required Components:**

- Laptop (**You need to bring your own laptop. I would recommend bringing laptops for all the group members as it will be a teamwork. At least 2 laptop is mandatory**).

**Task details:**

1. Using Wokwi, design the sensor node and its computing board by following the wiring and configuration steps given in the lab manual.
2. Configure an MQTT broker and define the topics the node will publish to and subscribe from, so that the device and the application layer can exchange messages.
3. Build an interactive dashboard in Node-RED that establishes bidirectional communication with the node. The dashboard should display the incoming sensor readings live, and should also allow instructions to be sent back to the device, for example to change a setpoint or to actuate an output.

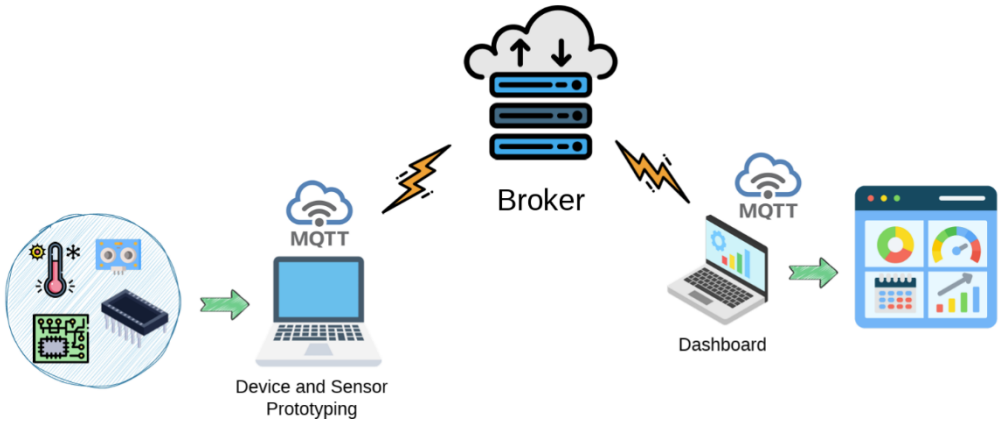
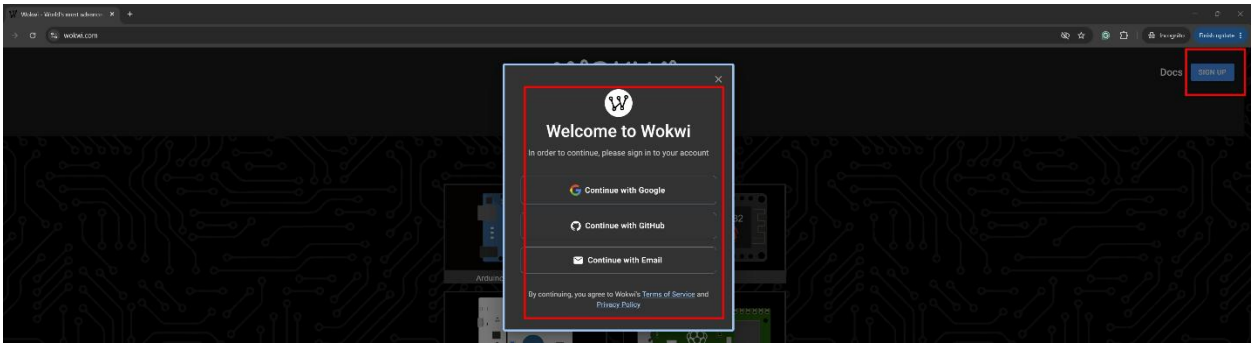


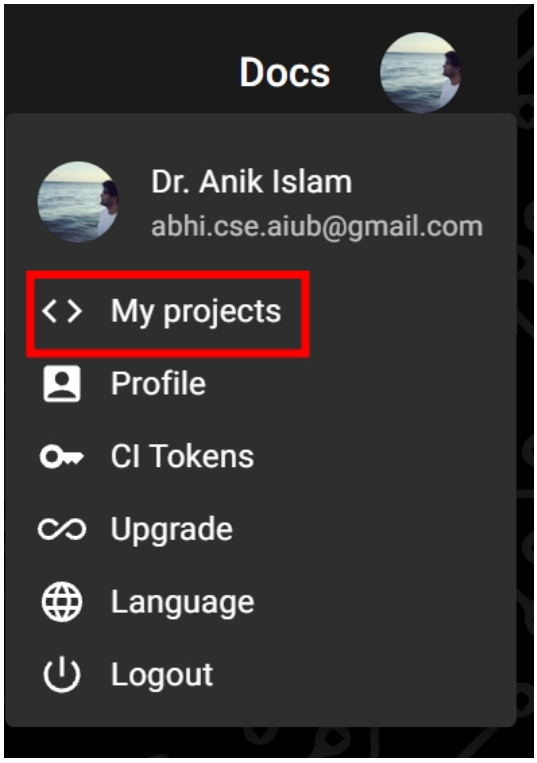

Figure 1 System design of the proposed task.

**<u>Hardware prototyping:</u>**

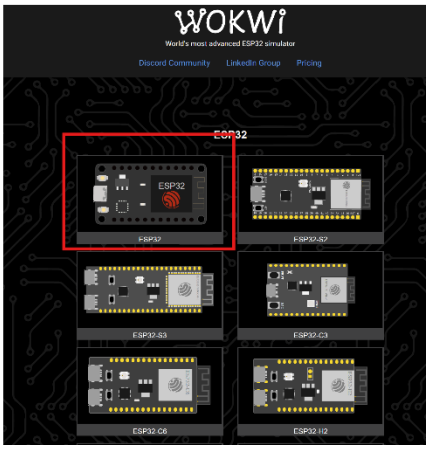
The prototyping will be carried out in Wokwi. Wokwi is an advanced, browser-based simulator for embedded systems and IoT projects. It lets you design, code, and test your electronics projects entirely online, so you do not need any physical hardware to get started. Everything runs right inside your web browser, including real time code execution, circuit wiring, and debugging. To begin, open Wokwi by visiting this link: https://wokwi.com/. Once the page loads, go ahead and create an account using the steps described below.



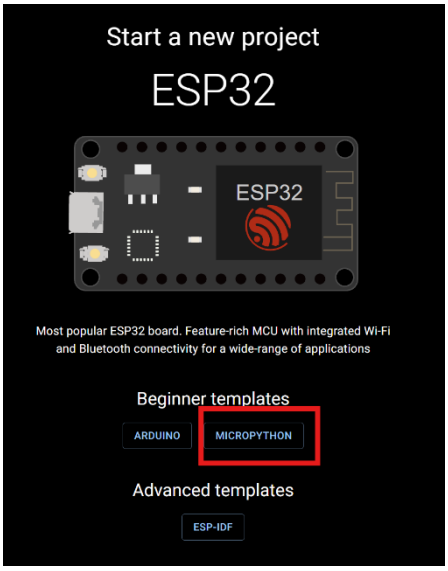

Once you have finished registering, log into the system. After that, head over to your projects as shown below.



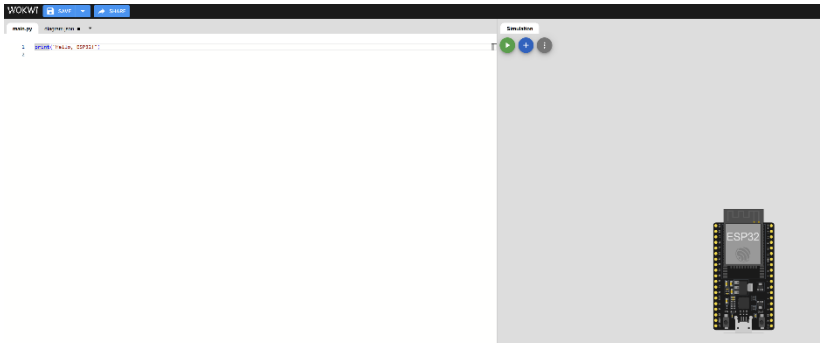

Next, click the "**New Project**" button to start a blank project. Once you have done that, select **ESP32** from the list of boards, as shown below. The **ESP32** is a microcontroller. In Lab Experiment 1, you worked with a Raspberry Pi. The ESP32 is similar in spirit, just a lighter weight version.



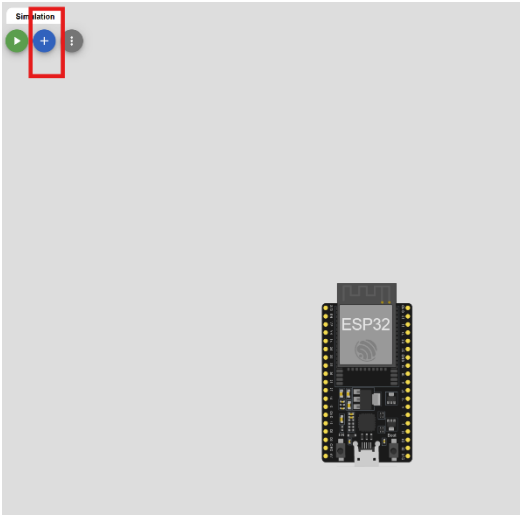
Next, it will ask you to pick a template. Go ahead and choose **MicroPython**, as shown below.



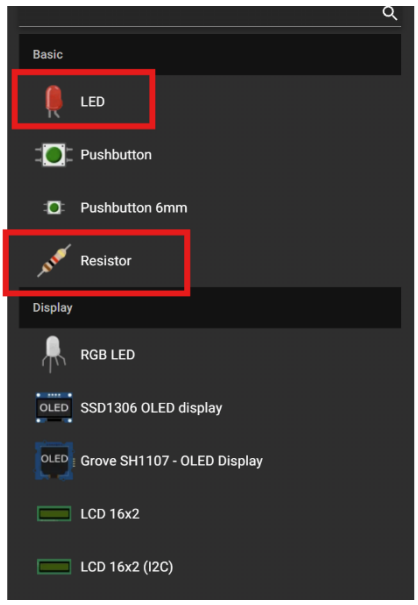

Once you have made your selection, the screen will look like this.



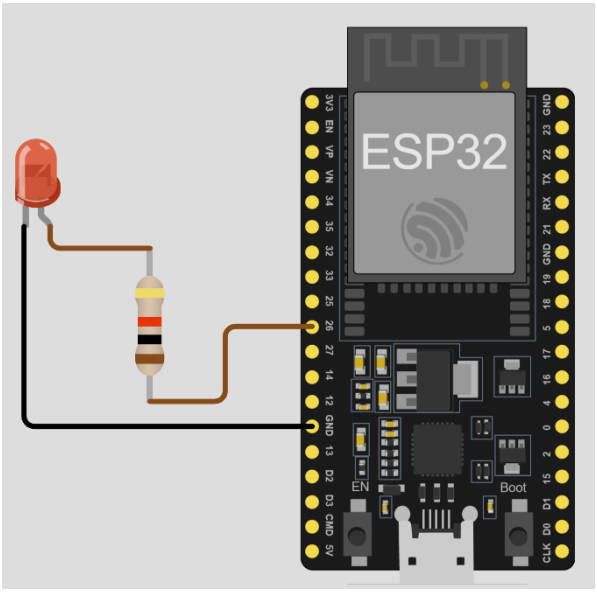

Next, press **Ctrl+s** to save your project. Give it the name **CPS_[your group number]** (for example, CPS_GROUP_10). While you are here, clear out the existing code in the **main.py** file on the **left** side. Once that is done, click the **+** button, as shown below.



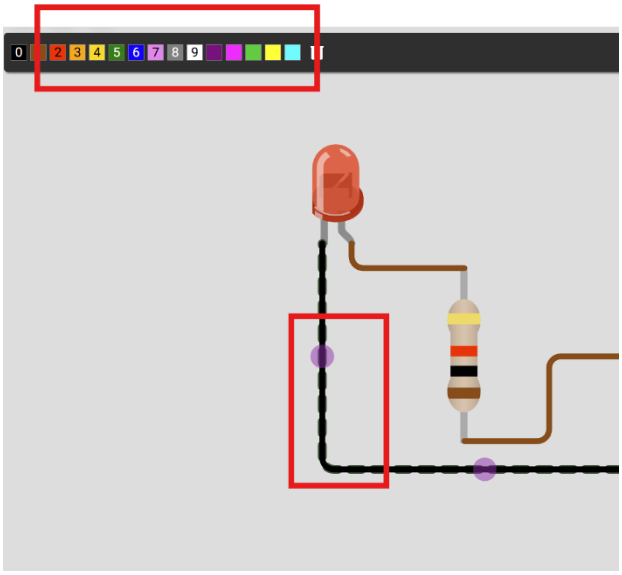
Next, pick out an **LED** light and a **resistor**, as shown below.



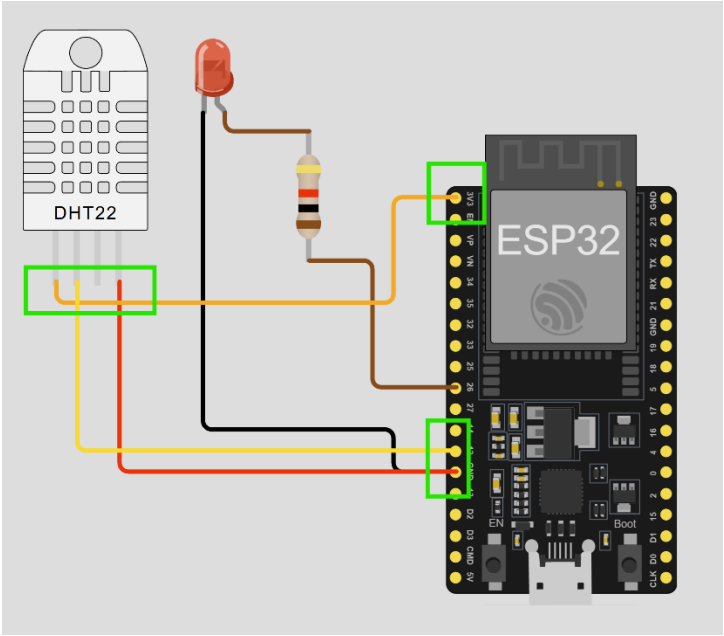

Next, connect one cable from the **LED** to the **GND** point on the **ESP32**, and run another cable through the resistor to pin **26**, as shown below.



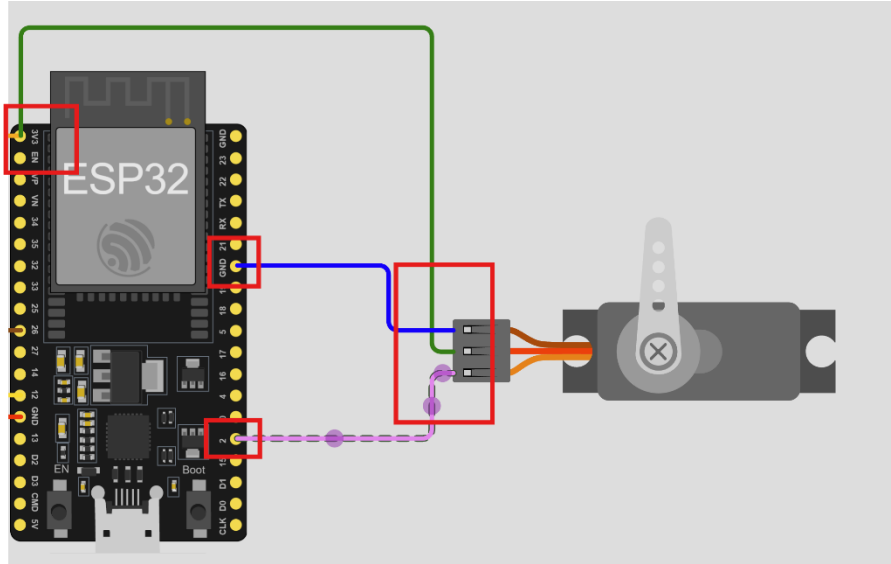

Try to give each connection its own unique color. You can change a cable's **color** simply by clicking on it, as shown below.



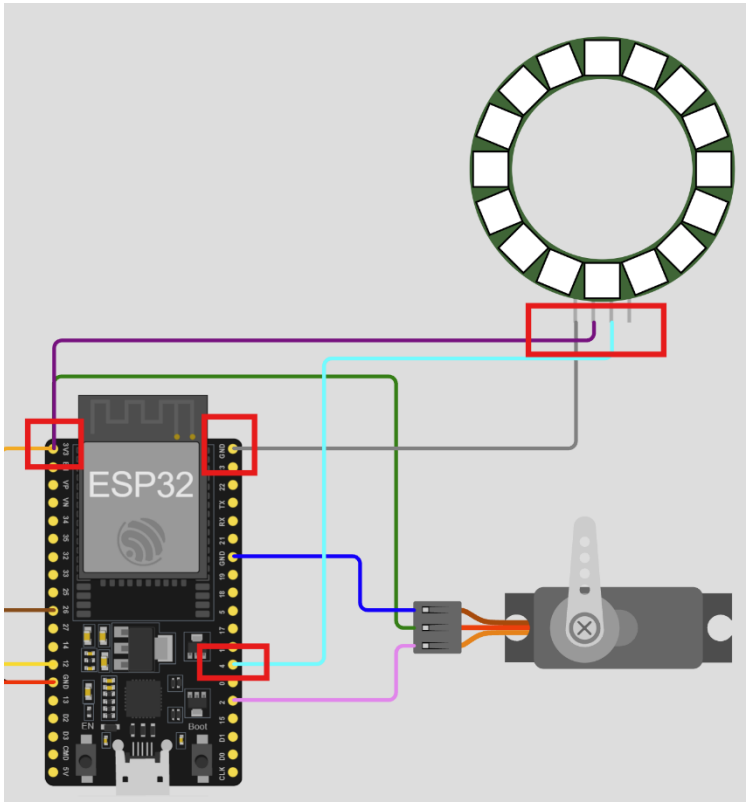
Next, choose the **DHT22** sensor from the list. If you hover your mouse pointer over any of the sensor's pins, it will show you what each pin stands for. Start by dragging a cable from the sensor's **GND** point to the ESP32's **GND** point. I marked this one **red**. After that, connect another cable from the sensor's **VCC** point to the board's **3V3** point, which I kept **orange**. Finally, draw one more cable from the sensor's **SDA** point to pin **12** on the board. I kept this cable **yellow**, as shown below.




Next, drag a servo motor from the list. Once it is in place, connect a cable from the motor's **V+** point to the board's **3V3** point. I kept this one green. After that, run a cable from the motor's **GND** point to the board's **GND** point, which I colored blue. Finally, draw a cable from the motor's **PWM** point to pin **2** on the board. I kept this cable light pink, as shown below.




Next, drag a NeoPixel compatible LED from the menu. Start by connecting a cable from the LED's **GND** to the board's **GND**, which I colored gray. After that, run a cable from the LED's **VCC** to the board's **3V3** point. I kept this one purple. Finally, draw a cable from the LED's **DIN** to pin **4** on the board. I kept this cable sky blue, as shown below.




Next, place the following code into the main.py file located on the left.

```
import network                          # WiFi networking
import time                             # Timing and delays
import dht                              # DHT temperature/humidity sensor driver
from machine import Pin, PWM            # GPIO pin and PWM control
import neopixel                         # WS2812 addressable LED driver
from umqtt.simple import MQTTClient     # Lightweight MQTT client

# ===== Pin Definitions =====
DHT_PIN = 12        # Data pin for the DHT22 sensor
LED_PIN = 26        # Pin driving the plain LED
SERVO_PIN = 2       # PWM pin driving the servo
NEO_PIN = 4         # Data pin for the NeoPixel strip
NUM_LEDS = 16       # Number of LEDs on the NeoPixel strip

# ===== MQTT & WiFi Configuration =====
WIFI_SSID = "Wokwi-GUEST"               # Wokwi's built in open network
WIFI_PASS = ""                          # No password for Wokwi-GUEST
MQTT_SERVER = "broker.hivemq.com"       # Public MQTT broker
CLIENT_ID = "ujaisldaaasdfgh_laslksdja1"  # Unique client ID for this device

# ===== Topic Configuration =====
# Change the group number here only, every topic below rebuilds automatically
group_number = "group_10"
BASE_TOPIC = f"cps/topic/{group_number}"      # Common prefix for all topics

TOPIC_PUB = f"{BASE_TOPIC}/Tempdata"          # Device publishes temp/humidity
here
TOPIC_LIGHTS = f"{BASE_TOPIC}/lights"         # Subscribe: LED on/off
TOPIC_SERVO = f"{BASE_TOPIC}/servo"           # Subscribe: servo angle
TOPIC_NEOPIXEL = f"{BASE_TOPIC}/lights/neopixel"  # Subscribe: NeoPixel color
# ================================

# ===== Initialize Hardware =====
led = Pin(LED_PIN, Pin.OUT)   # Configure LED pin as output
led.value(0)                  # Start with LED off

# DHT22 Setup
dht_sensor = dht.DHT22(Pin(DHT_PIN))   # Create the sensor object

# NeoPixel WS2812 Setup
np = neopixel.NeoPixel(Pin(NEO_PIN), NUM_LEDS)   # Create the LED strip object

# Servo Setup (50Hz PWM)
servo_pwm = PWM(Pin(SERVO_PIN), freq=50)   # Servos expect a 50Hz signal
```

```
def set_servo_angle(angle):
    """
    Maps 0-180 degrees to pulse width (500us to 2400us)
    matching ESP32Servo attach(SERVO_PIN, 500, 2400)
    """
    min_us = 500    # Pulse width for 0 degrees
    max_us = 2400   # Pulse width for 180 degrees
    # Linearly scale the requested angle to a pulse width in microseconds
    us = min_us + (angle / 180.0) * (max_us - min_us)
    # Convert microseconds to a 16 bit duty value (20,000us period at 50Hz)
    duty = int((us / 20000.0) * 65535)
    servo_pwm.duty_u16(duty)   # Apply the duty cycle

# Set initial servo position to 0 degrees
set_servo_angle(0)

def setup_wifi():
    # Connect the board to WiFi in station mode
    wlan = network.WLAN(network.STA_IF)
    wlan.active(True)                # Turn on the WiFi interface
    if not wlan.isconnected():
        print("Connecting to WiFi...")
        wlan.connect(WIFI_SSID, WIFI_PASS)
        # Wait in a loop until the connection is established
        while not wlan.isconnected():
            print(".", end="")
            time.sleep(0.5)
    print("\nWiFi connected")
    print("IP address: ", wlan.ifconfig()[0])   # Show the assigned IP

def mqtt_callback(topic, msg):
    # Runs every time a subscribed topic receives a message
    topic_str = topic.decode("utf-8")   # Convert topic bytes to string
    data_str = msg.decode("utf-8")      # Convert payload bytes to string

    # Debug print of the incoming message
    print(f"Message arrived in topic: {topic_str}")
    print(f"Message: {data_str}")
    print(f"Message size: {len(data_str)}")
    print("-----------------------")
    print(data_str)

    # Handle LED control messages
    if topic_str == TOPIC_LIGHTS:
        if data_str == "ON":
```

```
            print("LED")
            led.value(1)     # Turn LED on
        else:
            led.value(0)     # Turn LED off for anything other than "ON"

    # Handle servo angle messages
    elif topic_str == TOPIC_SERVO:
        try:
            degree = int(data_str)   # Payload should be a whole number
            print(f"Moving servo to degree: {degree}")
            set_servo_angle(degree)
        except ValueError:
            print("Invalid degree received")   # Payload was not a valid integer

    # Handle NeoPixel color messages
    elif topic_str == TOPIC_NEOPIXEL:
        try:
            # Parse CSV payload into RGB integers
            red, green, blue = map(int, data_str.split(","))
            print(f"Setting NeoPixel color to (R,G,B): {red},{green},{blue}")
            np.fill((red, green, blue))   # Apply color to all LEDs
            np.write()                    # Push the color data to the strip
        except ValueError:
            print("Invalid color payload. Expected format: R,G,B")

# Connect Network
setup_wifi()

# ===== Setup MQTT Client =====
client = MQTTClient(CLIENT_ID, MQTT_SERVER)   # Create the client
client.set_callback(mqtt_callback)            # Register the message handler

def reconnect():
    # Keep trying to connect and subscribe until it succeeds
    while True:
        try:
            print("Connecting to MQTT broker...")
            client.connect()
            # Subscribe to all three control topics
            client.subscribe(TOPIC_LIGHTS)
            client.subscribe(TOPIC_SERVO)
            client.subscribe(TOPIC_NEOPIXEL)
            print("MQTT connected")
            print("Topic Subscribed")
            break   # Exit the retry loop once connected
```

```
        except Exception as e:
            print(f"failed, error: {e} try again in 5 seconds")
            time.sleep(5)   # Wait before the next attempt

reconnect()

# ===== Main Loop timing =====
previous_millis = time.ticks_ms()   # Timestamp of the last publish
interval = 1000  # 1000 ms non-blocking delay

while True:
    try:
        # Non-blocking check for incoming MQTT messages
        client.check_msg()
    except OSError:
        reconnect()   # Reconnect if the connection dropped

    current_millis = time.ticks_ms()
    # Only publish once per interval without blocking the loop
    if time.ticks_diff(current_millis, previous_millis) >= interval:
        previous_millis = current_millis   # Reset the timer

        try:
            dht_sensor.measure()               # Trigger a new sensor reading
            temp = dht_sensor.temperature()    # Read temperature in Celsius
            hum = dht_sensor.humidity()        # Read humidity in percent

            print(f"Temperature: {temp:.1f}°C")
            print(f"Humidity: {hum:.1f}%")

            # Build a CSV payload "temp,hum," and publish it
            msg_str = f"{temp:.1f},{hum:.1f},"
            print(f"PUBLISH DATA: {msg_str}")
            client.publish(TOPIC_PUB, msg_str.encode("utf-8"))

        except Exception as e:
            print("Error reading temperature/humidity!")   # Sensor read failed

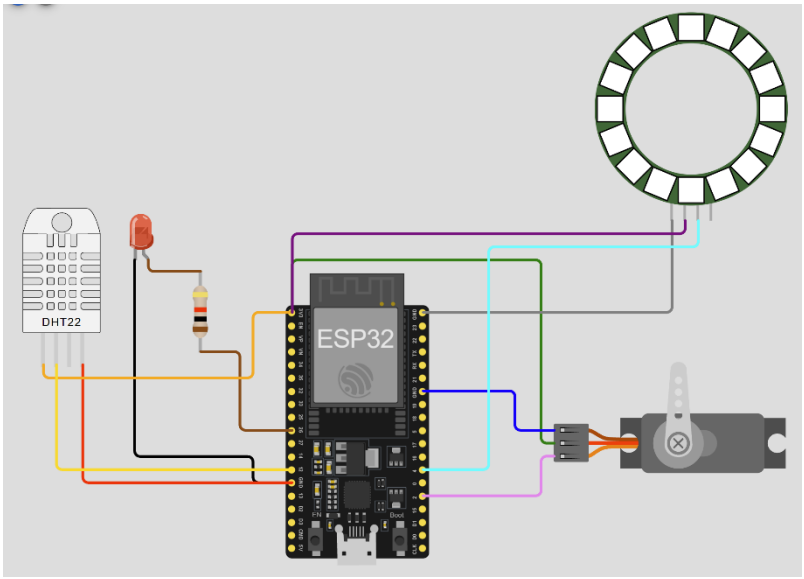
    time.sleep_ms(10)  # Short sleep yield to keep system stable
```

For this project, we have gone with a public cloud MQTT server, namely **HiveMQ** (link: https://www.hivemq.com/mqtt/public-mqtt-broker/). In the line that reads **'group_number = "group_10"'**, you need to enter your own group number. For instance, if your number is **10**, the line

becomes '**group_number = "group_10"**'. Putting it all together, the topic name for group 10 will look like this.

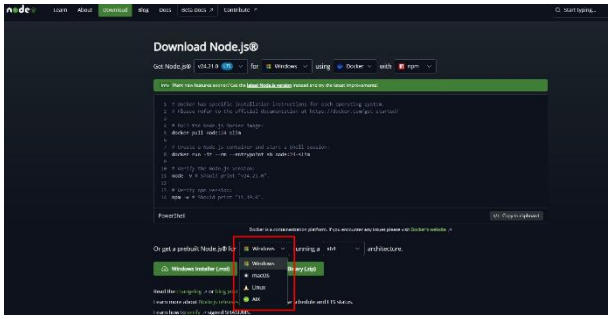
| Purpose | Topic |
|---|---|
| **Publish temperature and humidity** | cps/topic/group_10/Tempdata |
| **LED control** | cps/topic/group_10/lights |
| **Servo control** | cps/topic/group_10/servo |
| **NeoPixel color** | cps/topic/group_10/lights/neopixel |

Do not forget to update this in **Wokwi**. You will also need it later during the dashboard design part in Node-RED. The finished prototype will look like the one shown below.

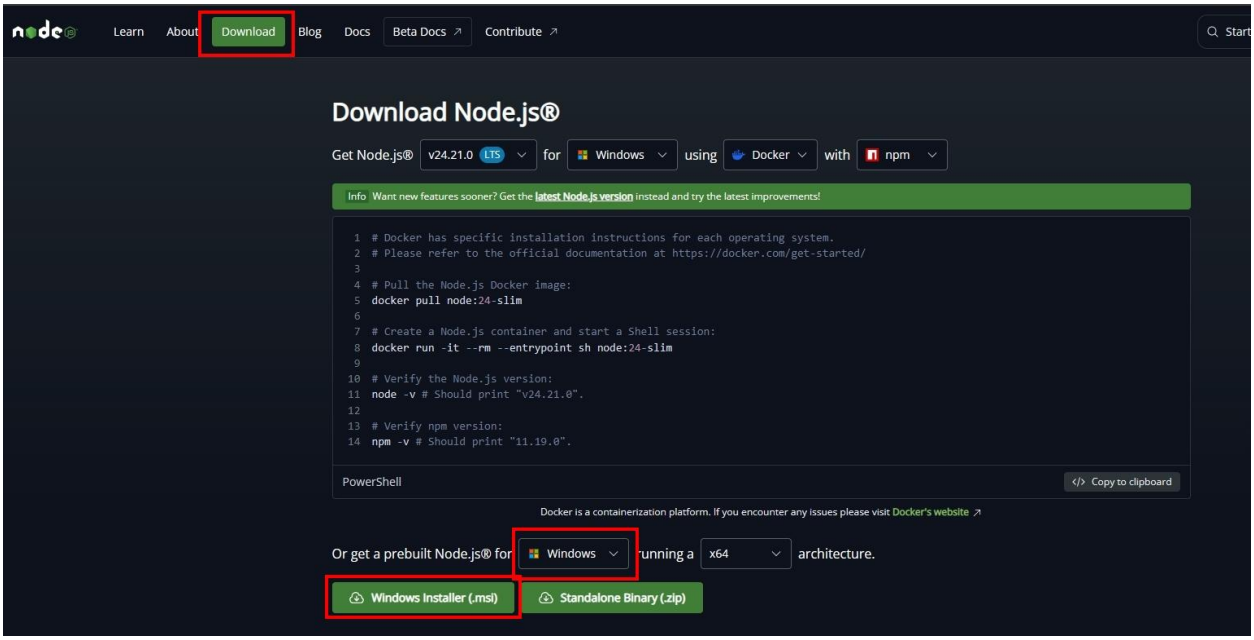



**<u>Dashboard Design:</u>**

To build the dashboard, we will be using **Node-RED**. Node-RED is a flow based, low code development tool for visual programming. It was originally created by IBM to wire together hardware devices, APIs, and online services for the Internet of Things. It offers a flow editor that runs right in your web browser, which you can use to put together JavaScript functions. Before we can install Node-RED, we first need to install **Node.js**. To do that, head over to the official download page at https://nodejs.org/en/download. On the download section, you will find support for several different operating systems, as shown below.



Pick the option that matches your platform. For example, if you are a **Windows** user, the options will look like this.



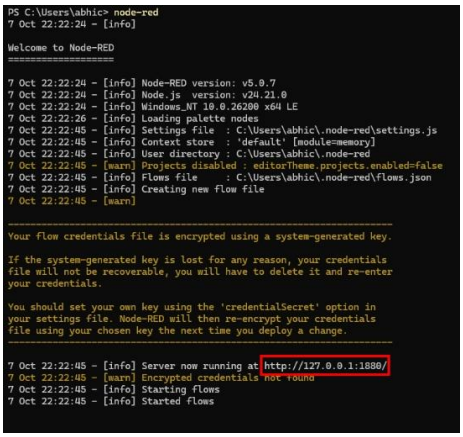

Once the installer file has downloaded, follow the prompts and install Node.js through the graphical installer. After the installation finishes, open your terminal or command prompt and type the command "**node -v**". If everything installed correctly, it will display the current version of Node.js. Next, type **npm install -g node-red** into the terminal or command prompt. There are also other ways to install it, which you can explore here: [https://nodered.org/docs/getting-started/local](https://nodered.org/docs/getting-started/local). Once you enter the command, it might look something like this.

```
C:\Users\abhic>npm install -g node-red

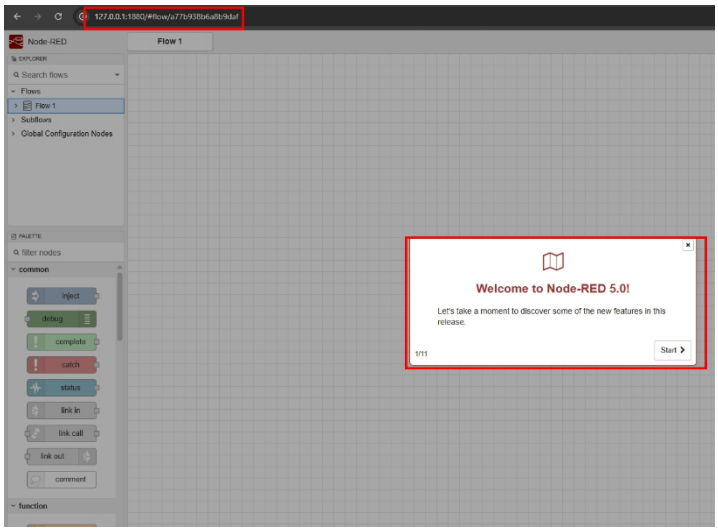
added 277 packages in 32s

75 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New major version of npm available! 11.19.0 -> 12.2.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.0
npm notice To update run: npm install -g npm@12.2.0
npm notice
```

Next, to start Node-RED, type "**node-red**" and it will give you a link, as shown below. Copy that link and paste it into your browser.



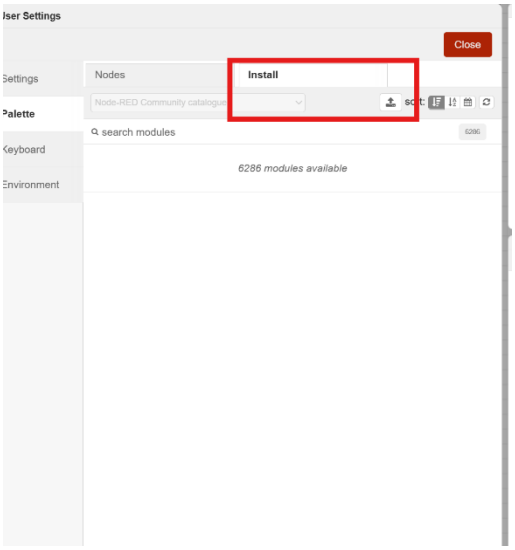
Once you have pasted the link, the view will look like this.



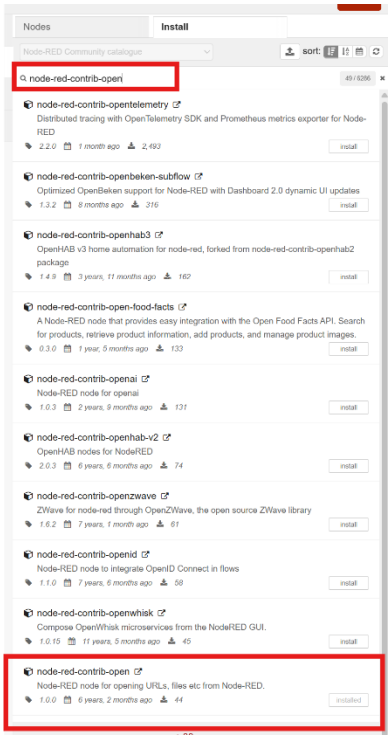

Next, we need to install a few palettes for the dashboard. To get to the palette manager, follow the instructions below (**menu icon > Manage Palette**).



Next, click on the **Install** tab, as shown below.



Using the search box, type **node-red-contrib-open** and **install** it, as shown below.




In the same way, search for the following and install it:

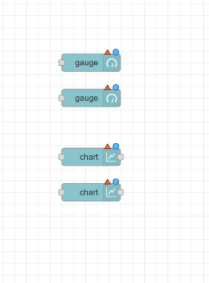
- **node-red-dashboard**

After the installation is complete, you will see the dashboard UI related nodes appear, as shown below.



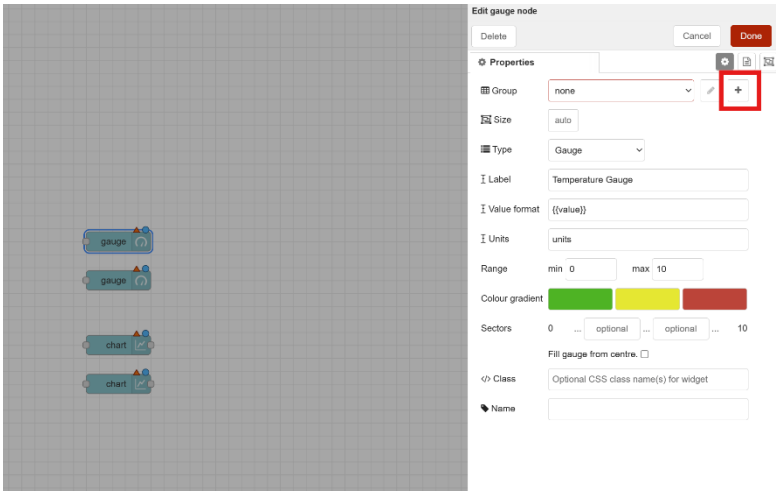
In the node-red, the access manager appears as follows:
Flow design board: http://127.0.0.1:1880/#flow

UI: http://127.0.0.1:1880/ui/

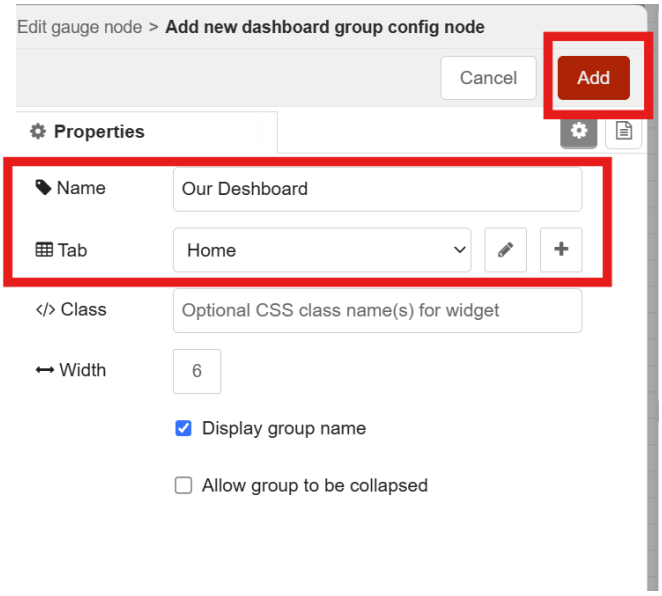



From the dashboard palette, drag and drop two gauges and two charts, as shown besides.

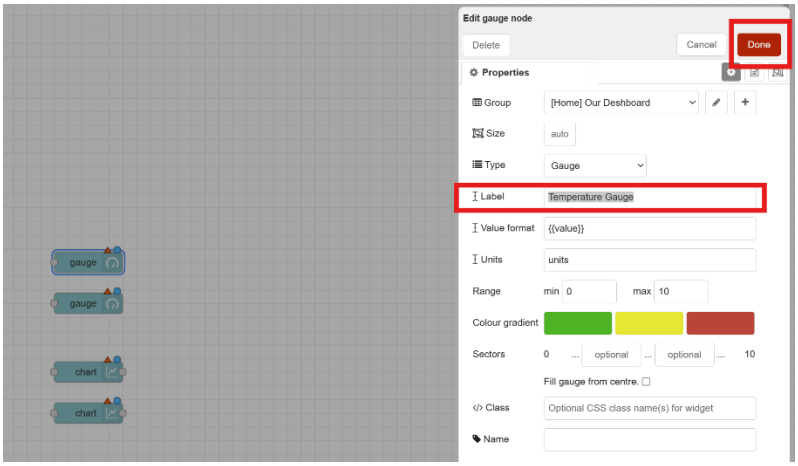
You will need to rename these nodes. To do this, start by double clicking on the first gauge. A popup window will appear, as shown below.



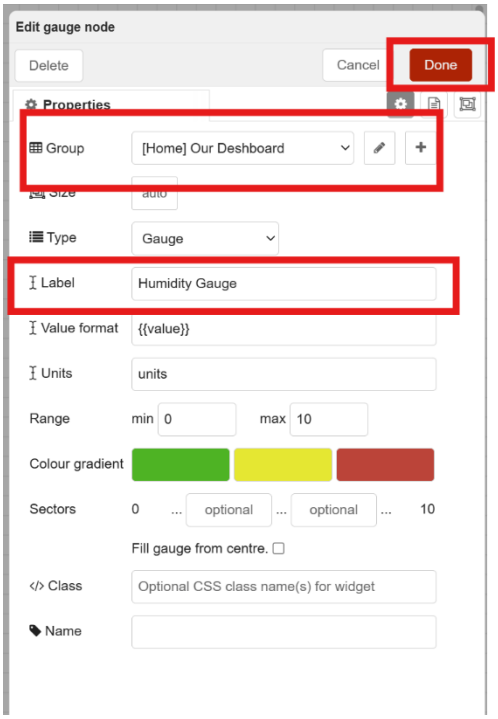

From this popup, we need to create a group. To do that, click on the icon marked with a **+**. Another popup will come forward. In that window, type "**Our Dashboard**" into the Name field and select **Home** from the Tab option. Finally, press the **Add** button. The overall view will look like this.



Once you press the Add button, it will take you back to the previous popup. From there, change the label to "**Temperature Gauge**" and press **Done**, as shown below.



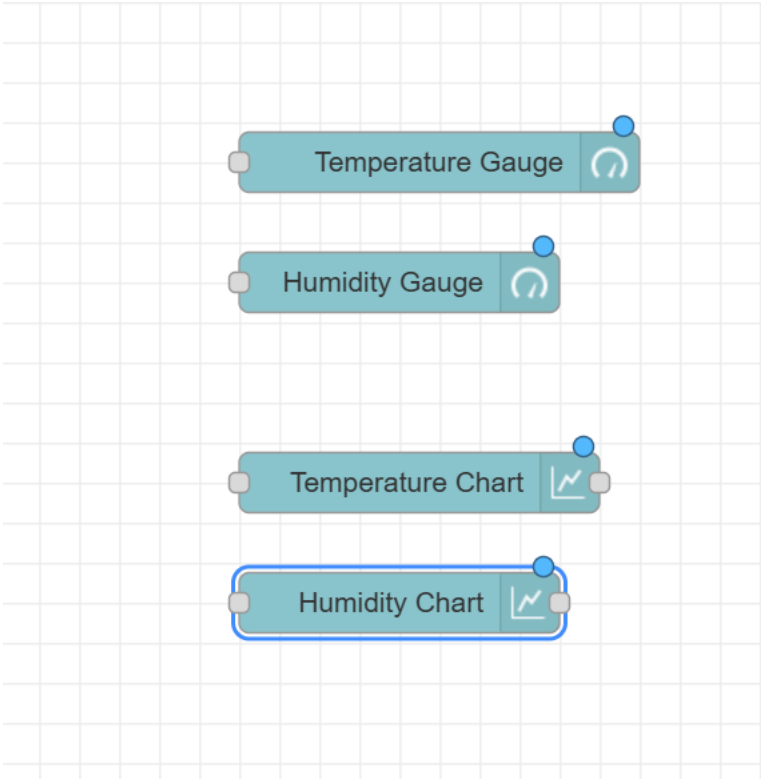

For the next gauge, double click on it. Then select the group from the menu, change the label to "**Humidity Gauge**", and press **Done**. The view will look like this.



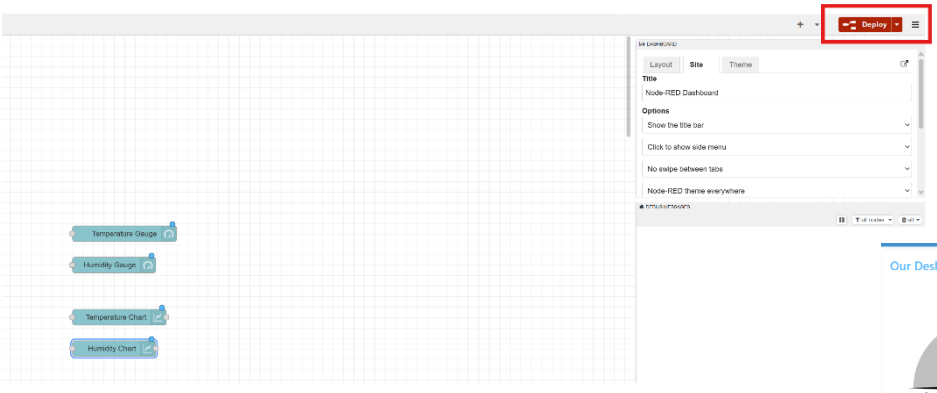

The process for the charts is the same. Here is what you need to do:

- Rename chart1 to "**Temperature Chart**", choose the **Home** group from the dropdown, and press **Done**
- Rename chart2 to "**Humidity Chart**", choose the **Home** group from the dropdown, and press **Done**

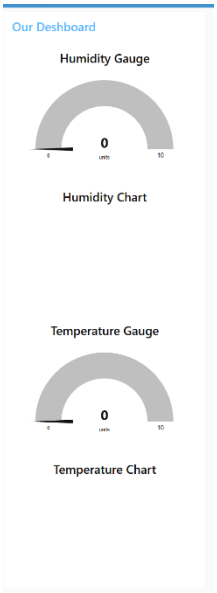
Once you have made these changes, it will look like this:



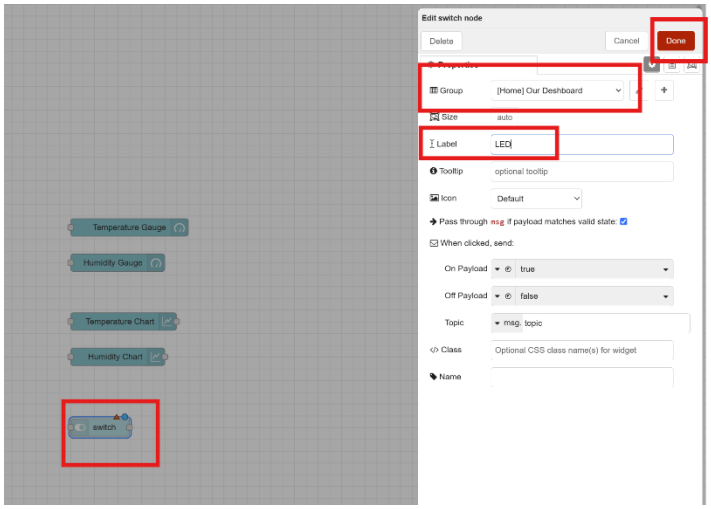

Next, press **Deploy** to apply the changes to the UI, as shown below.



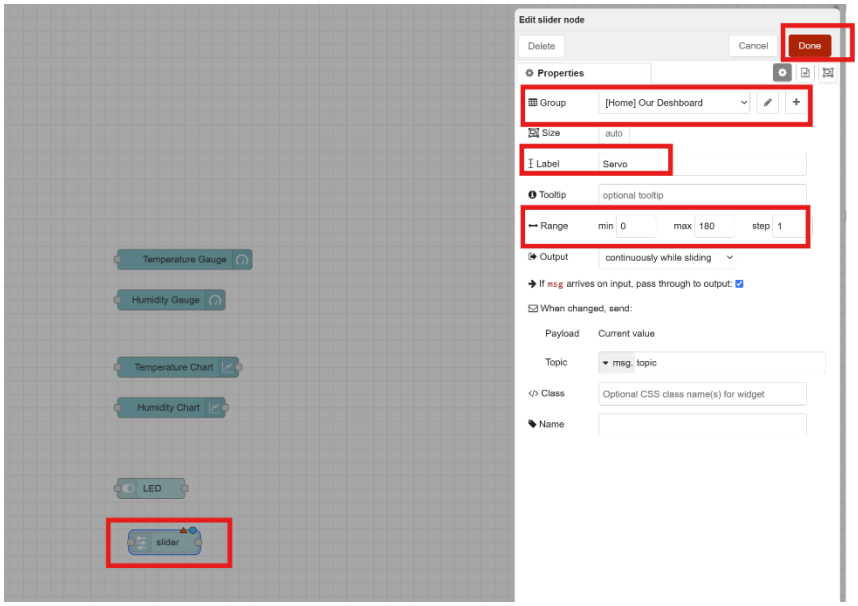

To view the UI, go to the URL [http://127.0.0.1:1880/ui/](http://127.0.0.1:1880/ui/) and the view will look like this.



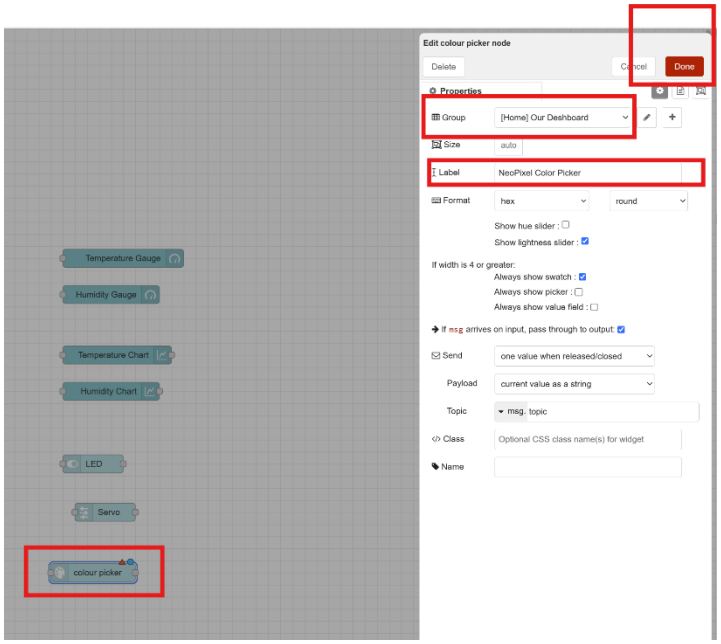
Next, drag a switch from the dashboard palette, change the label to “**LED**”, and select the group from the dropdown. Finally, press **Done**. The view will look like this.



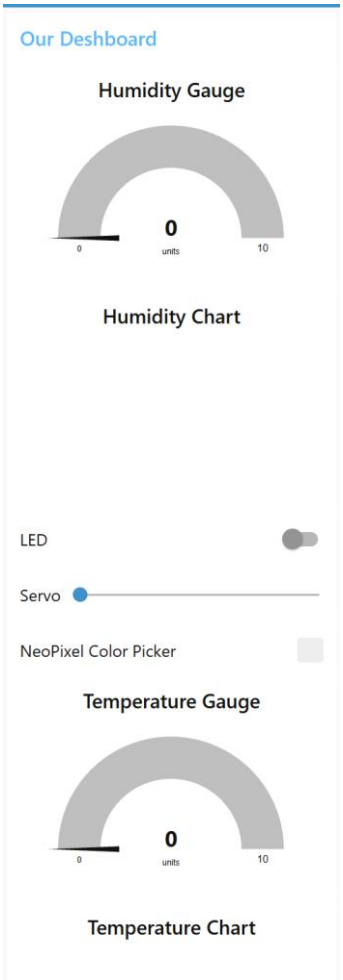

Next, drag a **slider** from the same palette and double click on it. Rename it to “**Servo**”, choose the **Home** group from the dropdown, and set the maximum range to **180**. Once that is done, press **Done**. The view will look like this.



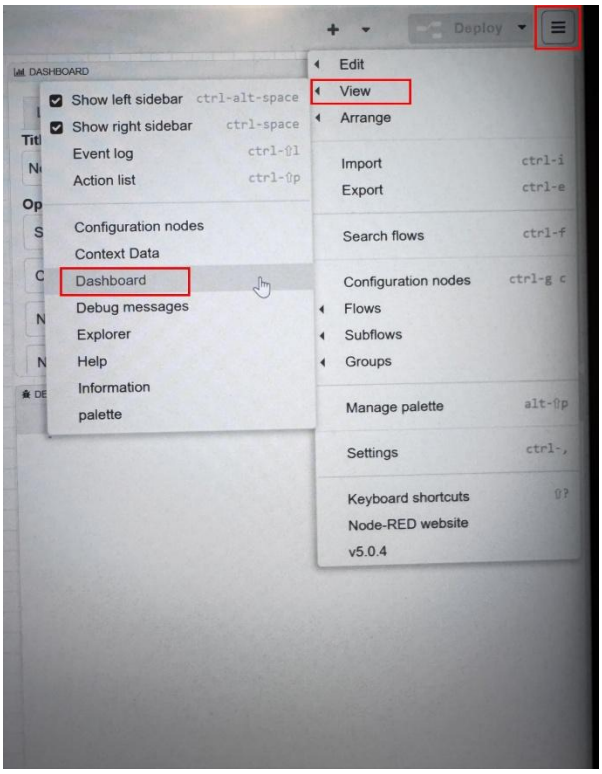

Next, drag a color picker onto the canvas and rename it to “**NeoPixel Color Picker**”. The view will look like this.



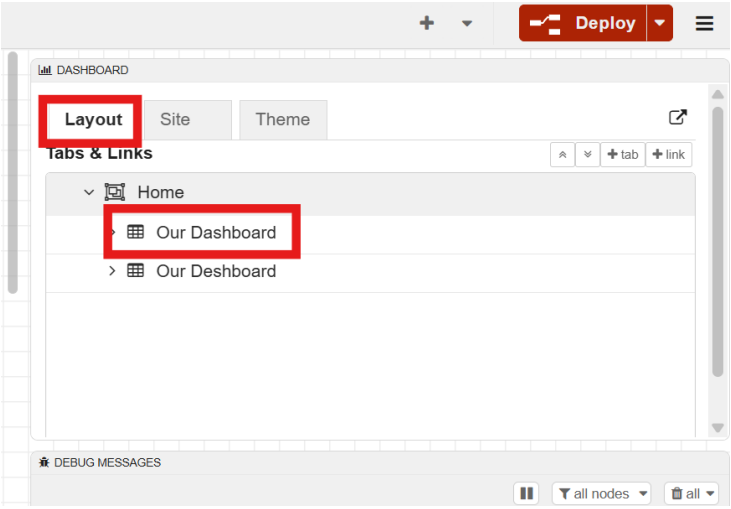

Next, press Deploy and the UI will look like this.



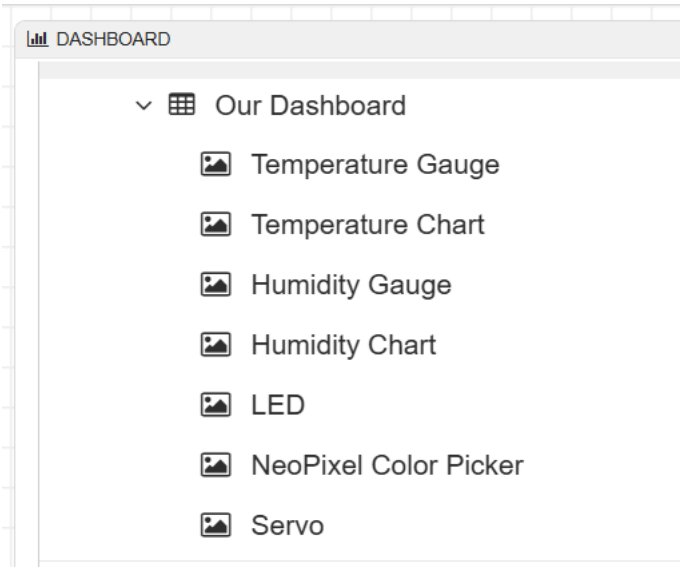
We need to rearrange the controls. To do this, click on the **menu** icon, which is located next to the **Deploy** button. From there, go to **View > Dashboard**. The view will look like this.



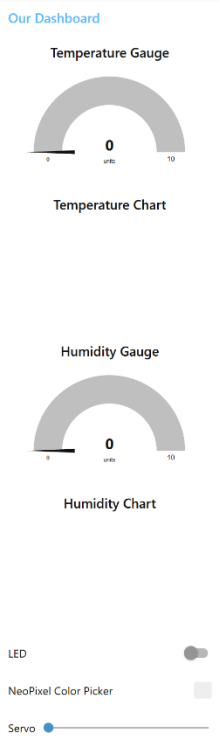

Next, you will see a **Layout** tab on the right. Click on "**Our Dashboard**", as shown below.



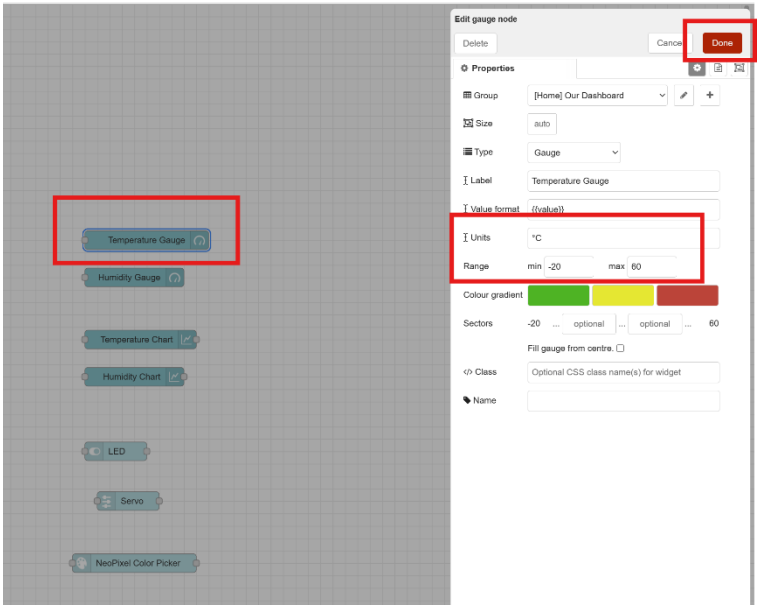

From there, rearrange the controls as shown below.



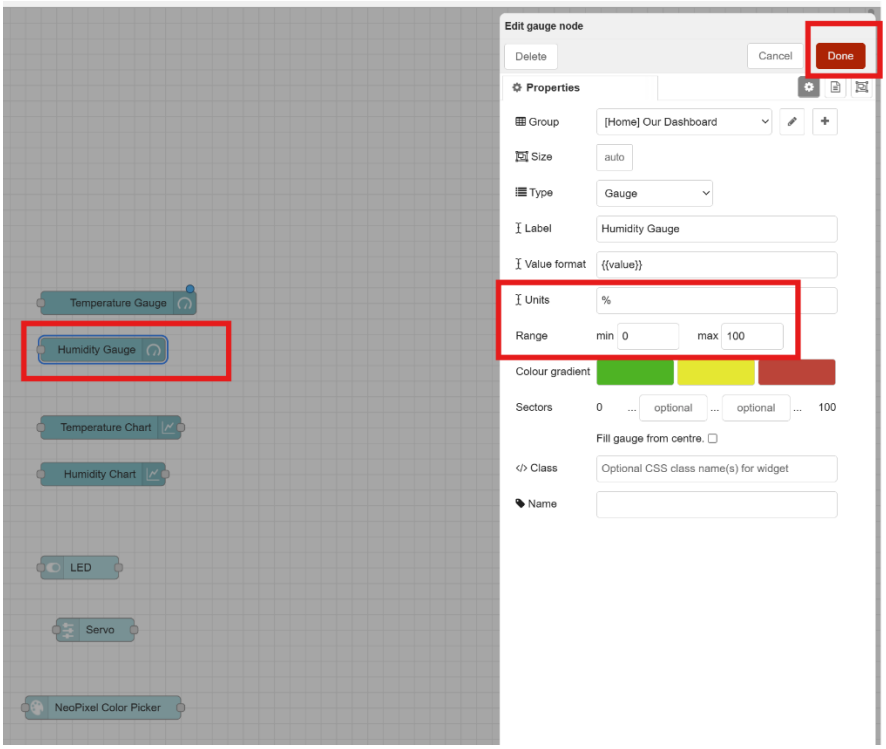

After making your changes, press the **Deploy** button and visit the UI to see the following view.



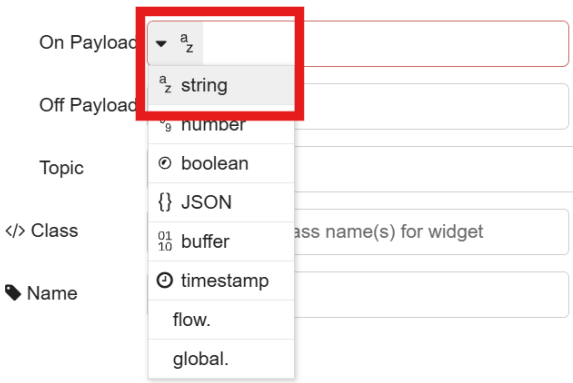

Now we need to adjust a few properties of the controls. Start by double clicking on the **Temperature Gauge**. Set the Units to **°C**, the minimum value to **-20**, and the maximum value to **60**, then press **Done**. The view will look like this.



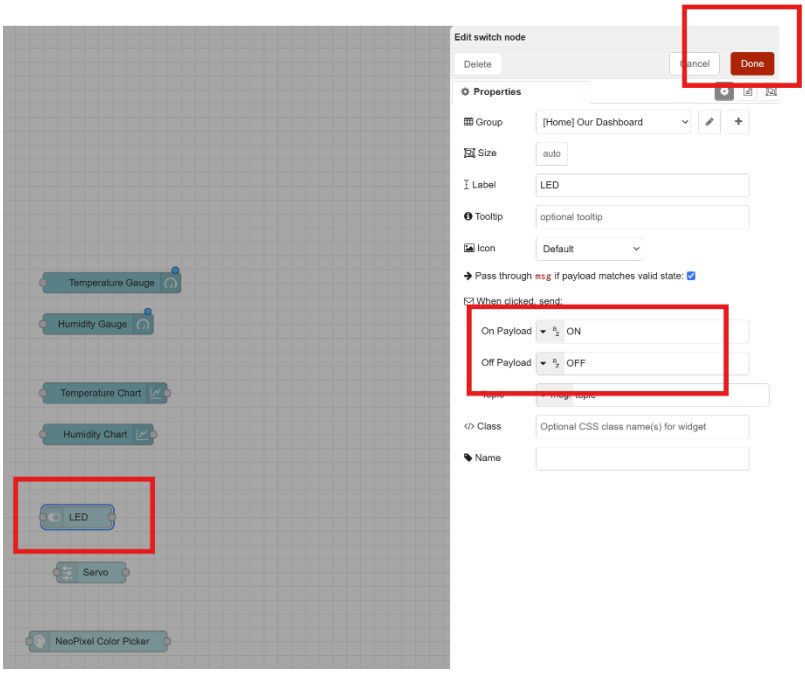

Next, double click on the Humidity Gauge. Set the Units to **%**, the maximum value to **100**, then press **Done**. The view will look like this.



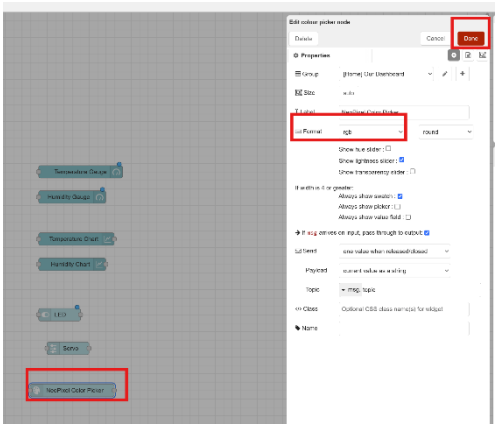

Next, double click on the **LED**. Change both the On Payload and Off Payload data types to **string**, as shown below.



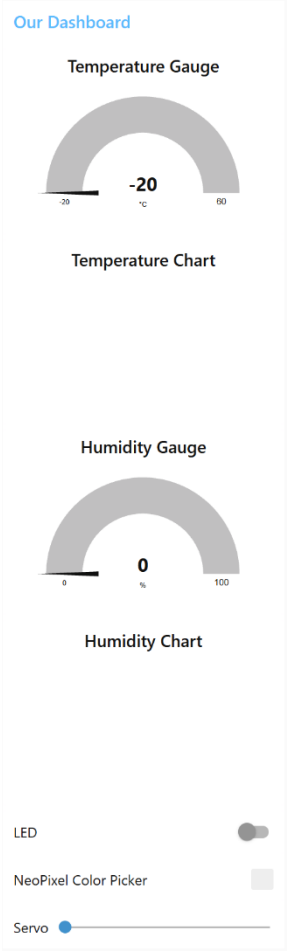

Next, set the On Payload value to **ON** and the Off Payload value to **OFF**. Once that is done, press the **Done** button. The view will look like this.



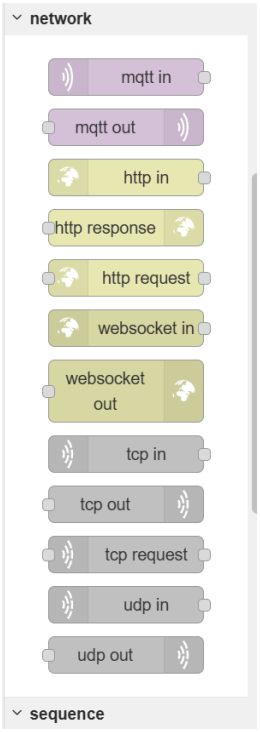

Next, double click on the NeoPixel and change the **Format** to **RGB**, as shown below.



Next, press **Deploy** and load the UI. The view will look like this.



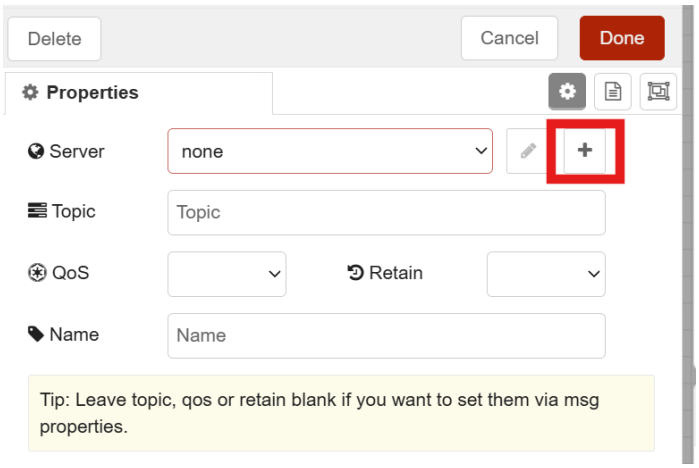

Now we need to connect to the prototype through MQTT. The nodes related to this connection can be found under the **Network** palette, as shown besides.



Now, start by dragging an **MQTT Out** node and placing it next to the **LED**. **MQTT Out** is used for sending data, while **MQTT In** is used for receiving data. Draw a cable from the **LED** and connect it to the **MQTT Out** node, as shown below.




Next, double click on the **MQTT** node. A popup will come forward. First, we need to set up the MQTT server. To do that, click on the **+** icon in the Server row, as shown below.

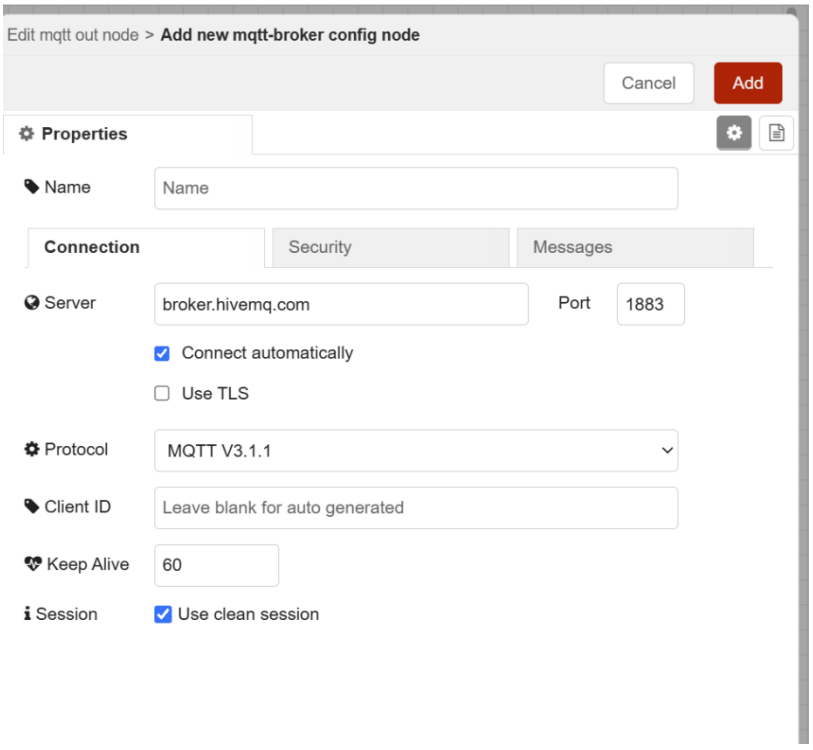



Once you press that button, another popup will appear. In that window, enter the following information and press **Add**.

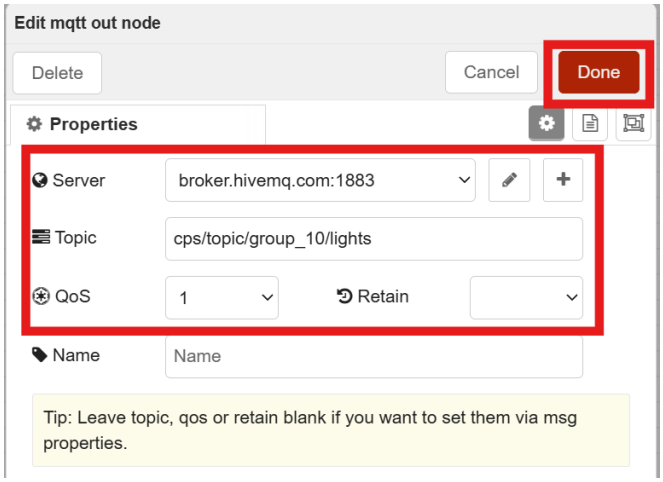
**Server**: broker.hivemq.com

**Port**: 1883

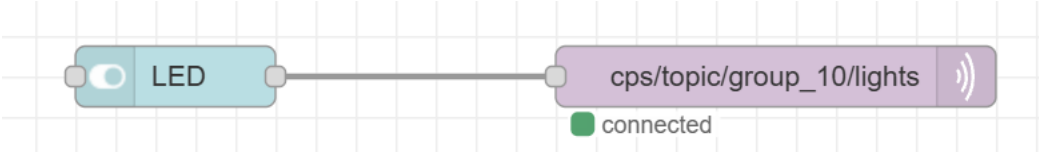
The view will look like this.



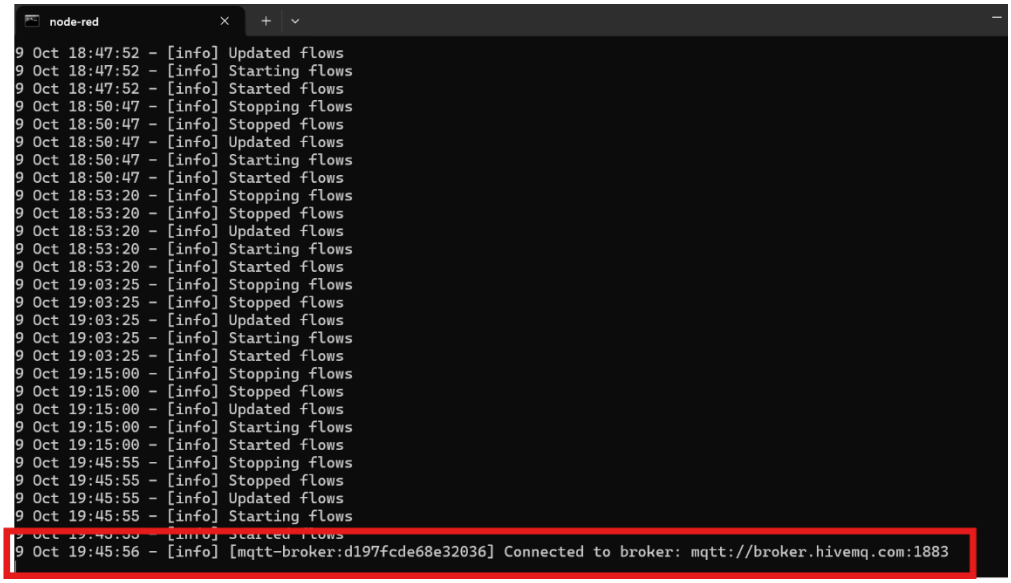
Next, it will take you back to the previous popup. In this window, set the topic to "**cps/topic/group_10/lights**" (**remember to replace the group ID with your own**) and set the QoS to **1**. Once that is done, press the **Done** button. The view will look like this.




Next, press **Deploy**. If everything is set up correctly, it will show as **connected**, as shown below.



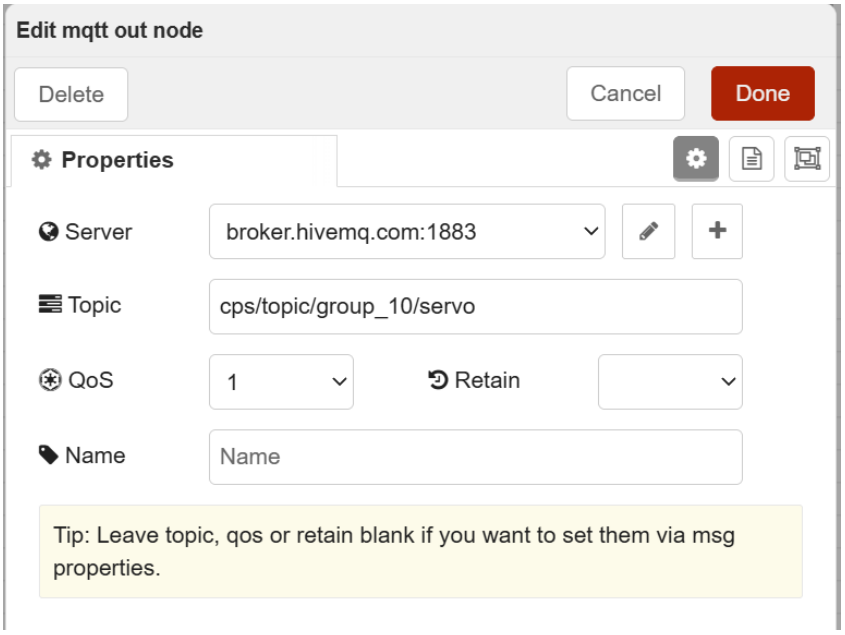

If you check your terminal or command prompt, you will see the connection message, as shown below.



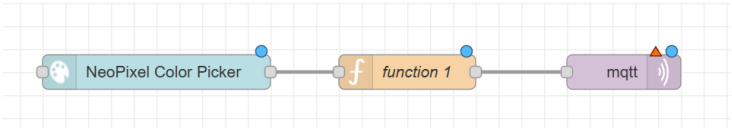

Next, drag another **MQTT Out** node and connect it to the **Servo**, as shown below.



Next, double click on the **MQTT** node that is connected to the **Servo**. In the popup, select the **server** from the menu, set the topic name to "**cps/topic/group_10/servo**" (**be sure to change the group number to match your own**), set the QoS to **1**, and press **Done**. The view will look like this.



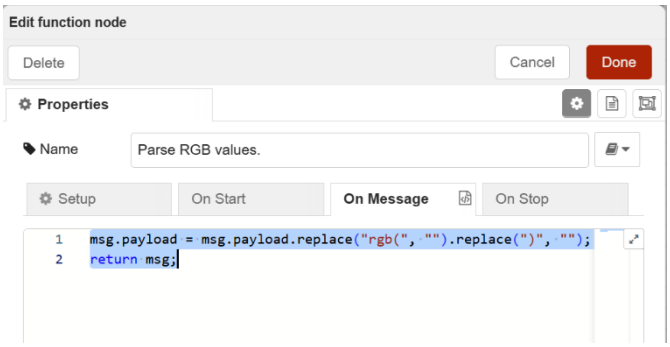

Now drag another **MQTT Out** node for the NeoPixel. The catch here is that the data the color picker provides does not match what the sensor expects, so we need to map it first. To do this, drag a **Function** node from the **Function** palette. The view will look like this.



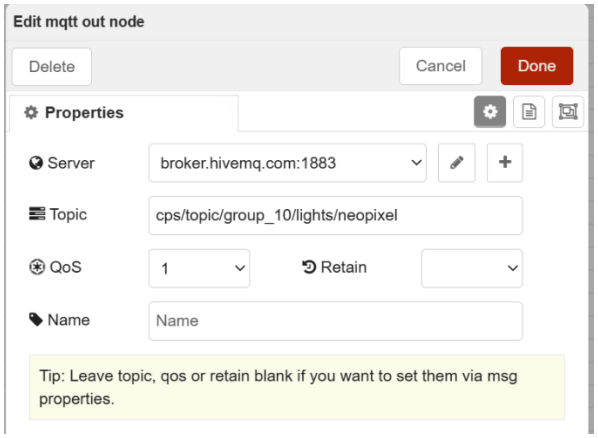

After that, double click on the function and go to its **On Message** tab. You need to write the following code.

```
msg.payload = msg.payload.replace("rgb(", "").replace(")", "");
return msg;
```

After that rename it to "**Parse RGB values**." And press **done** button. The views will be as follows.



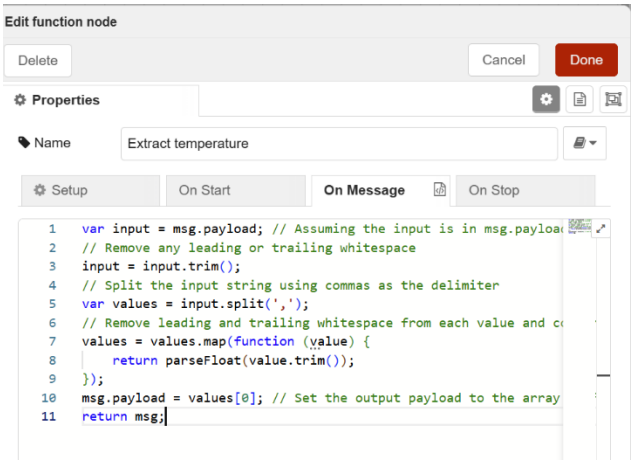
Next, double click on the **MQTT** node that is connected to the **NeoPixel**. In the popup, select the **server** from the menu, set the topic name to "**cps/topic/group_10/lights/neopixel**" (**be sure to change the group number to match your own**), set the QoS to **1**, and press **Done**. The view will look like this.



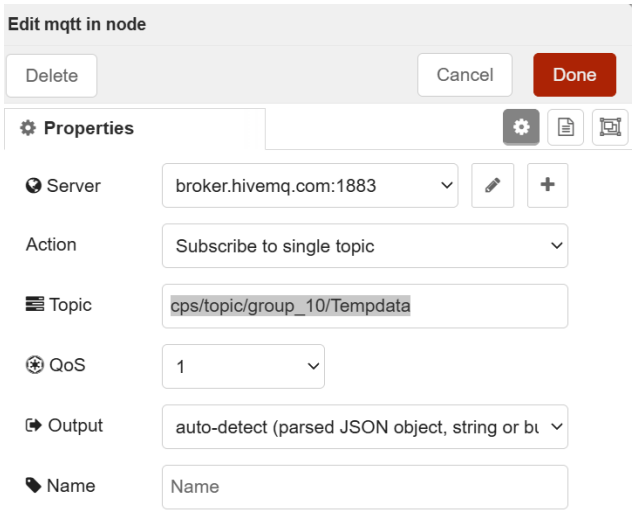

Now, bring in an **MQTT In** node to display the temperature data. The sensor sends its readings as comma separated values, so we will need a function to map that data once again. Drag a **Function** node and place it in between the two. Double click on the **function**, rename it to "**Extract temperature**", and put the following code into the On Message tab.

```
var input = msg.payload; // Assuming the input is in msg.payload
// Remove any leading or trailing whitespace
input = input.trim();
// Split the input string using commas as the delimiter
var values = input.split(',');
// Remove leading and trailing whitespace from each value and convert to float
values = values.map(function (value) {
    return parseFloat(value.trim());
});
msg.payload = values[0]; // Set the output payload to the array of float values
return msg;
```

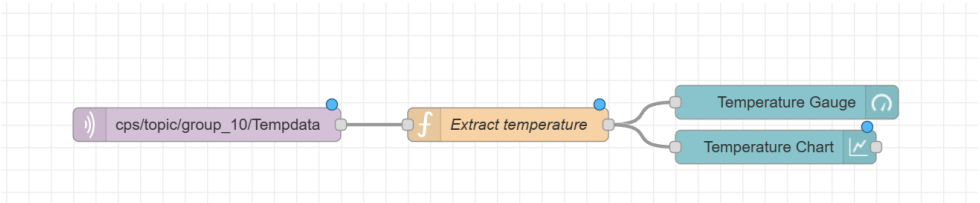
Next, press **Done**. The view will look like this.



Next, double click on the **MQTT In** node. Select the **server** from the dropdown, set the topic name to "**cps/topic/group_10/Tempdata**" (**be sure to change the group number to match your own**), set the QoS to **1**, and press Done. The view will look like this.



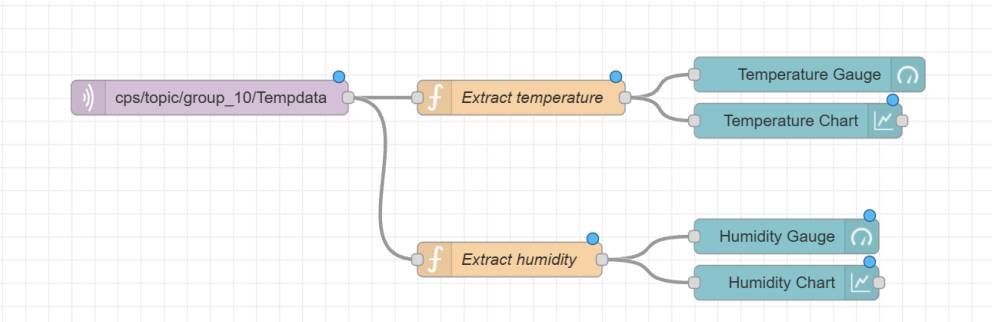

Next, connect the **MQTT In** node to the function, then connect the function to both the **Temperature Gauge** and the **Temperature Chart**, as shown below.



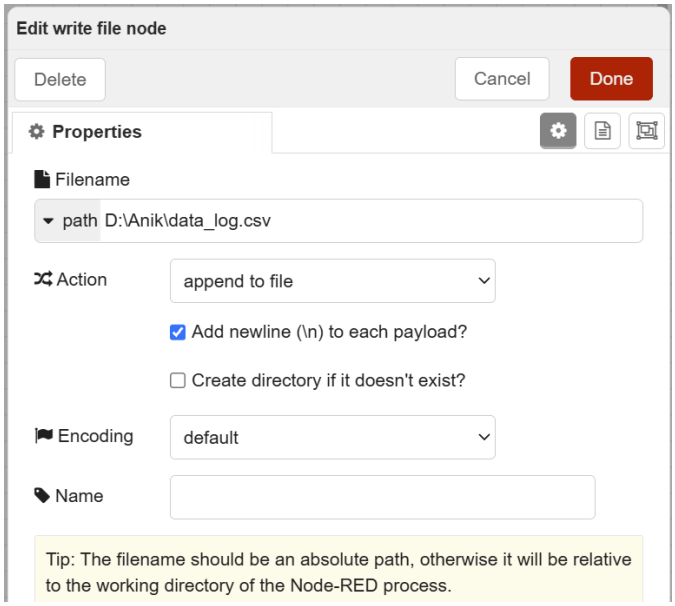

Next, drag another **Function** node, rename it to "**Extract humidity**", and add the following code.

```
var input = msg.payload; // Assuming the input is in msg.payload
// Remove any leading or trailing whitespace
input = input.trim();
// Split the input string using commas as the delimiter
var values = input.split(',');
// Remove leading and trailing whitespace from each value and convert to float
values = values.map(function (value) {
    return parseFloat(value.trim());
});
msg.payload = values[1]; // Set the output payload to the array of float values
return msg;
```

Next, connect the **MQTT In** node to the function, then connect the function to both the **Humidity Gauge** and the **Humidity Chart**, as shown below.




Now, we need to log the data from the temperature and humidity sensor onto the local PC. To do this, drag a **Write File** node from the **Storage** palette. Once it is placed, double click on it. In the path field, **create** a folder on your PC and enter its **path** there. For example, I created a folder on my **D** drive and entered it like this: "**D:\Anik\data_log.csv**". Once that is done, press the Done button. The view will look like this.



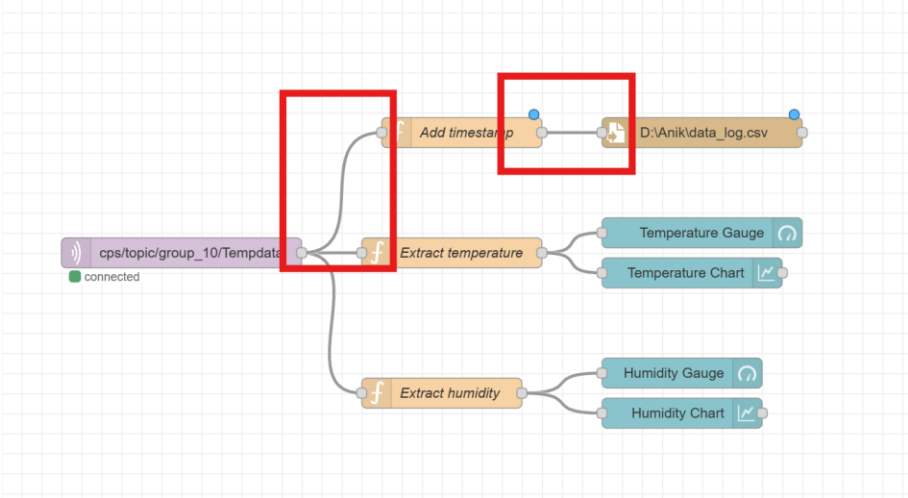

To log the data properly, we need to attach a **timestamp**, since the incoming data does not come with a time of its own. To do this, drag a **Function** node from the **Function** palette and double click on it. Rename it to "**Add timestamp**" and add the following code.

```
var timestamp = new Date().toISOString(); // Get the current timestamp in ISO
8601 format
var value = msg.payload; // Get the input value

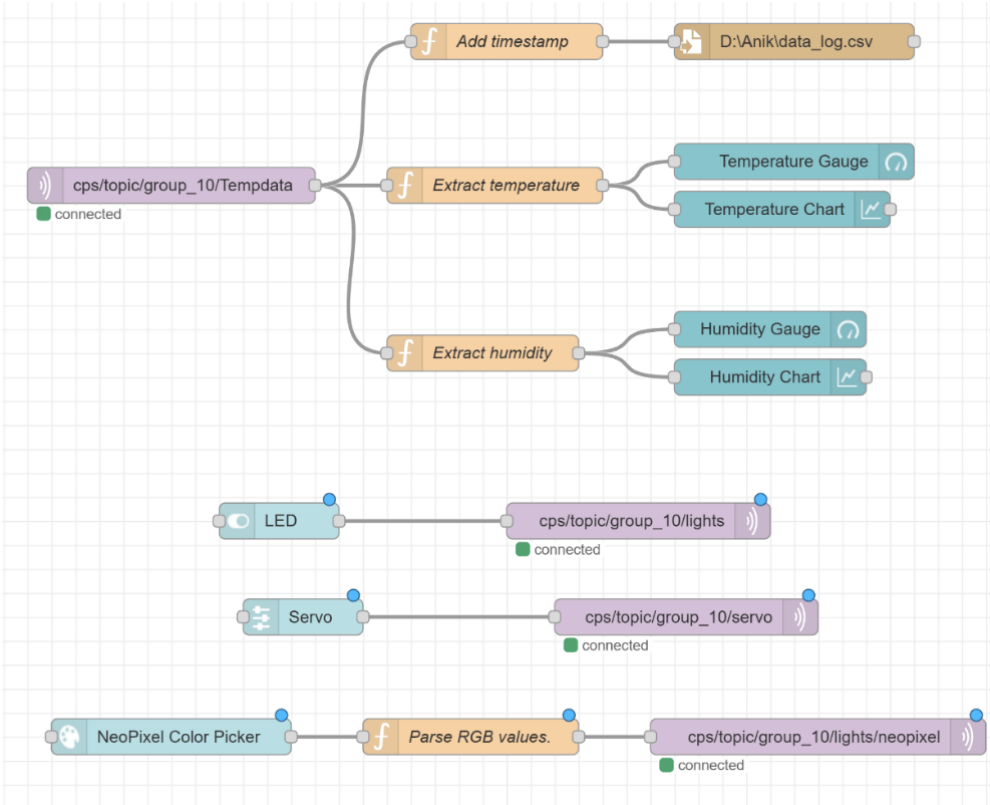
// Create the output message
msg.payload = timestamp + ", " + value;

return msg;
```

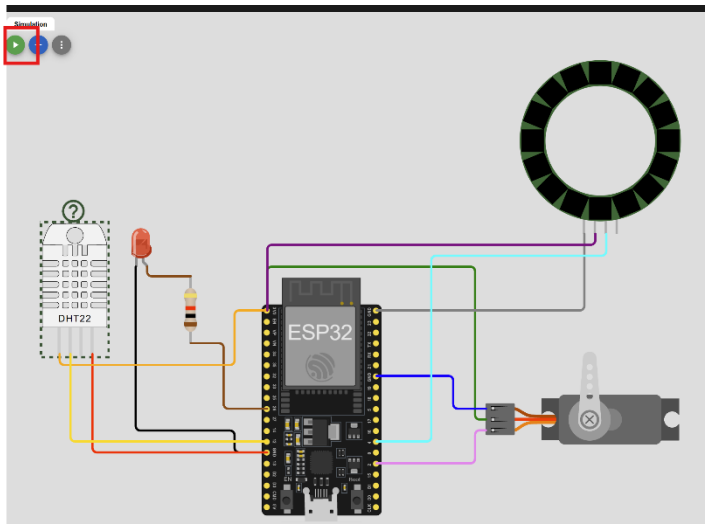
Next, connect the function to the **MQTT In** node and to the **Write File** node, as shown below.



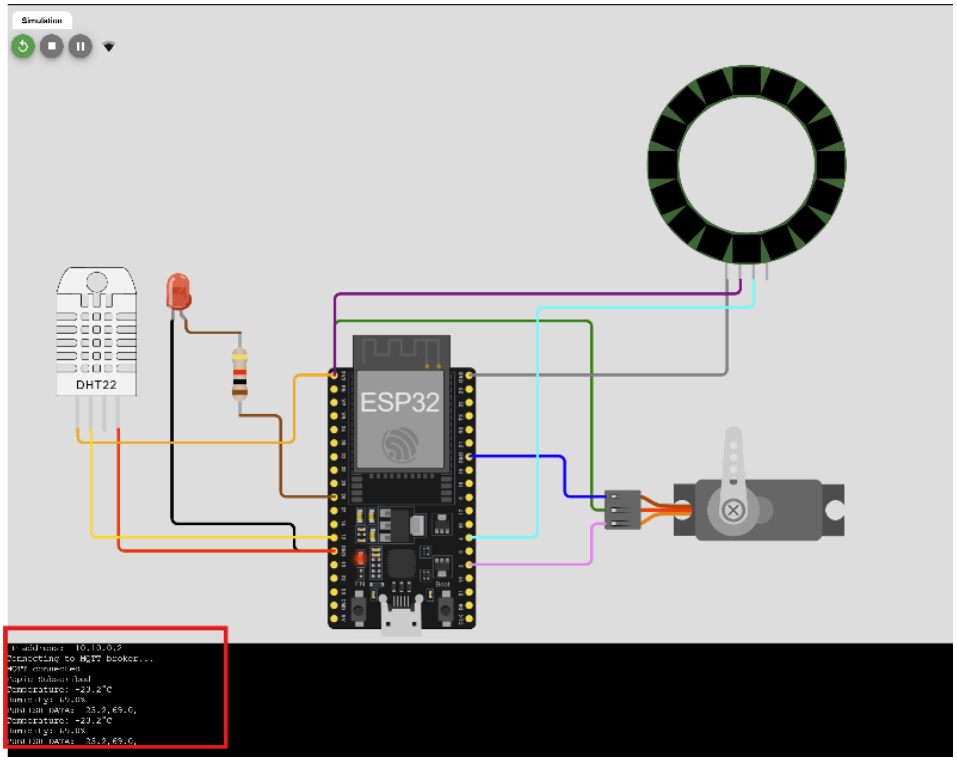

Next, press **Deploy**. The final view is shown below.



Now it is time to run the full project. To do this, first head over to **Wokwi** and press the "**Start Simulation**" button, as shown below.




If everything is working as it should, you will see the following.




Now, head over to your **Node-RED** UI and try playing with both the dashboard and the prototype side by side. You can watch this video (https://www.youtube.com/shorts/yPMiJUgkIiI) to get a feel for how to move around the prototype. After that, open the folder whose path you entered in the **Write File** node. Inside, you will find a file named "**data_log.csv**". This file contains all the data that has been saved from the sensors.

## Answer the following questions as part of your lab report.

1. What is Wokwi? What benefits does virtual prototyping provide before physical hardware is purchased?
2. What is MQTT? Explain the role of the broker, topic, publisher and subscriber. Why do we use publish and subscribe here instead of connecting the dashboard directly to the device?
3. Why does each group need a unique topic and a unique client ID on a public broker? What happens if two groups use the same one?
4. What is QoS in MQTT? What QoS levels exist? What are the drawbacks of using QoS 0?
5. The device publishes temperature and humidity as comma separated values, which is why you needed function nodes to extract each reading. What other payload formats could be used, and which would you choose for a real deployment? Why?
6. How did you verify that communication works in both directions? Present your results in figures or a table and include a sample of your logged data.

**Format of the final report:**

- **Introduction**
  - In this introduction, you will provide details about the task. For example, you will discuss what the task was about, how tasks were discussed among your group, how the task was divided, who did which tasks, and so on.
  - Which sensors, actuators and topic structure did you consider, and why did you consider them?
- **Experiments**
  - How did you perform the experiment (including expected screenshots of the Wokwi circuit, the Node-RED flow, and the dashboard)?
  - What difficulties did you face during the experiment, and how did you solve them?
  - How did teamwork assist you in completing the experiment?
- **Performance Evaluation**
  - Demonstrate that communication works in both directions. For the device to dashboard path, provide a figure or table comparing the value printed in the Wokwi serial monitor, the value displayed on the dashboard, and the value written to the log file at the same instant.
  - For the dashboard to device path, provide a figure or table showing each command sent (LED, servo angle, NeoPixel colour) and the observed response on the prototype.
  - **Record a video that demonstrates all the components working together, much like the YouTube video linked above**.
- **Code & dataset**
  - Upload the following to a cloud platform and share an active link: your Wokwi project (**export it by going to the menu and**

**clicking “Download Project ZIP”**), your exported Node-RED flow (**click the menu icon next to the Deploy button and then press Export**), your recorded video or a link to it hosted on YouTube or another cloud platform, and your logged data file. Just make sure the link stays active.

Overall, make sure to answer the provided questions, which are attached to each task, in their respective sections and ensure they are highlighted appropriately. Additionally, include each member’s name and ID accurately, along with a description of their individual contributions.